



UNIVERSITÀ DEGLI STUDI DI SALERNO
DIPARTIMENTO DI INFORMATICA
DIPARTIMENTO DI ECCELLENZA



CORSO DI LAUREA IN INFORMATICA

Tecnologie Software per il Web

HTML

PROF. ATTILIO DELLA GRECA

a.a. 2025-2026

Cos'è HTML?

HTML (HyperText Markup Language) è il linguaggio di markup standard per la creazione di pagine web. Non è un linguaggio di programmazione, ma un linguaggio che definisce la **struttura e il contenuto** di un documento web.

Gli elementi HTML etichettano parti di contenuto come "questo è un titolo", "questo è un paragrafo", "questo è un collegamento", ecc.

Markup Annotazione del contenuto tramite tag

Ipertesto Collegamento tra documenti via link

Standard W3C Mantenuto dal World Wide Web Consortium

Breve storia dell'HTML





Breve storia dell'HTML

HTML 1.0 – 1991 Tim Berners-Lee pubblica la prima versione informale di HTML, con appena 18 tag. Il primo sito web della storia, info.cern.ch, va online il 20 dicembre 1990 e descrive il progetto stesso del World Wide Web.

HTML 2.0 – 1995 Primo standard ufficiale, pubblicato dall'IETF (Internet Engineering Task Force) come RFC 1866. Formalizza ciò che i browser dell'epoca già implementavano in modo disordinato: testo, link, immagini e form di base. È il primo tentativo serio di mettere ordine nel caos delle implementazioni proprietarie di Netscape e Mosaic.

Breve storia dell'HTML

HTML 3.2 – 1997 Introduce il supporto per tabelle, applet Java, testo attorno alle immagini e i primi attributi di presentazione come font e colori. È il periodo della cosiddetta "guerra dei browser" tra Netscape Navigator e Internet Explorer, in cui ciascun produttore aggiunge tag proprietari non standard.

HTML 4.0 – 1997 Introduce esplicitamente la separazione tra struttura (HTML) e presentazione (CSS), introduce il supporto per i fogli di stile, lo scripting con JavaScript, i frame per dividere la finestra del browser e miglioramenti significativi all'accessibilità e all'internazionalizzazione.

HTML 4.01 – 1999 è una correzione di bug della 4.0 e rimarrà lo standard ufficiale per ben quindici anni.

Breve storia dell'HTML

XHTML 1.0 – 2000. L'idea è riformulare HTML secondo le regole rigide di XML: tutti i tag devono essere chiusi, tutto in minuscolo, attributi sempre tra virgolette, nesting rigoroso. XHTML 1.0 è sostanzialmente HTML 4.01 riscritto in sintassi XML.

WHATWG – 2004 Un gruppo di sviluppatori di Apple, Mozilla e Opera, frustrati dalla direzione del W3C, fonda il **WHATWG** (Web Hypertext Application Technology Working Group) e inizia a lavorare su una nuova specifica chiamata inizialmente "Web Applications 1.0", che diventerà **HTML5**. L'obiettivo è evolvere HTML in modo retrocompatibile, aggiungendo funzionalità per le applicazioni web moderne senza rompere i siti esistenti.

Breve storia dell'HTML

HTML 5.0 – 2014 Le novità sono enormi: nuovi elementi semantici, elementi multimediali nativi (che eliminano la dipendenza da Flash), nuovi tipi di input per i form, API per geolocalizzazione, drag & drop, Web Storage, e un DOCTYPE drasticamente semplificato.

HTML Living Standard – 2019 W3C e WHATWG firmano un accordo storico: da quel momento esiste **un'unica specifica HTML**, mantenuta dal WHATWG come Living Standard (pubblicata su html.spec.whatwg.org). Il W3C non pubblica più versioni separate. Questo significa che non ci sarà mai un "HTML6": le nuove funzionalità vengono aggiunte gradualmente allo standard corrente, i browser le implementano man mano, e gli sviluppatori devono mantenersi aggiornati consultando la documentazione ufficiale. Il web è diventato un organismo vivente, in costante evoluzione.

Struttura di un document HTML

```
<!DOCTYPE html>
<html lang="it">
  <head>
    <meta charset="UTF-8">
    <title>La mia pagina</title>
  </head>
  <body>
    <h1>Ciao Mondo!</h1>
    <p>Il mio primo sito.</p>
  </body>
</html>
```

<!DOCTYPE html>

Dichiara il tipo di documento (HTML5)

<html>

Elemento radice del documento

<head>

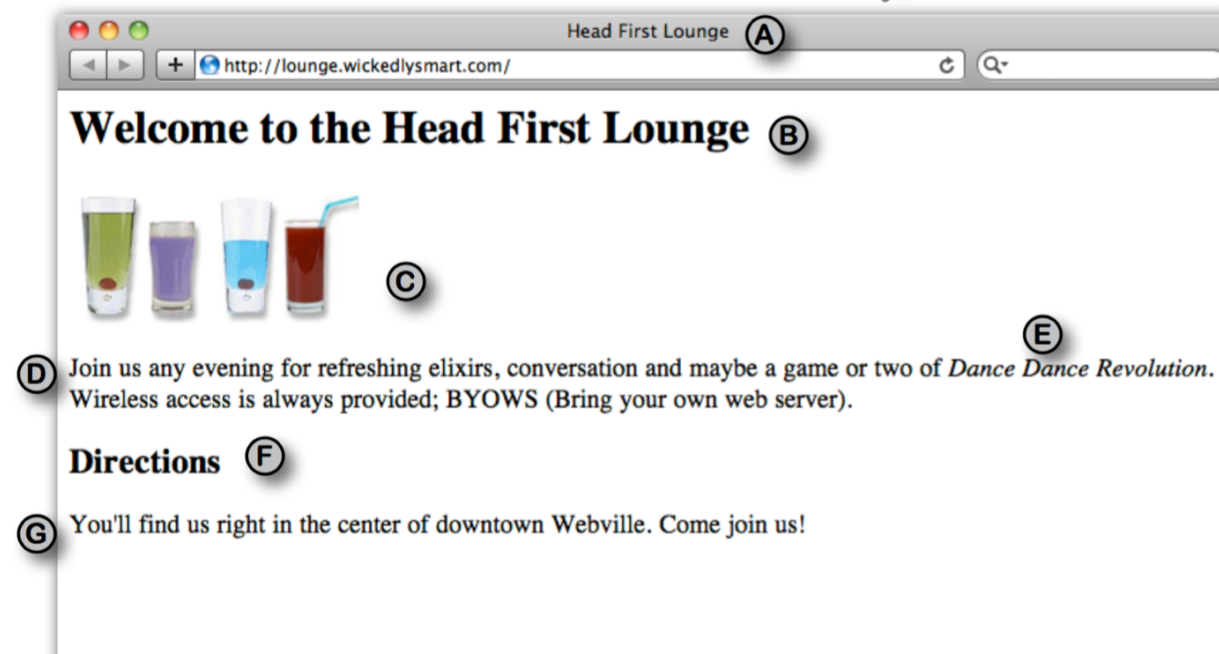
Metadati: titolo, charset, CSS, meta tag

<body>

Contenuto visibile della pagina


```
<html>
  <head>
    <title>Head First Lounge</title> (A)
  </head>
  <body>
    <h1>Welcome to the Head First Lounge</h1> (B)
     (C)
    <p>
      (D) Join us any evening for refreshing elixirs,
      conversation and maybe a game or
      two of <em>Dance Dance Revolution</em>. (E)
      Wireless access is always provided;
      BYOWS (Bring your own web server).
    </p>
    <h2>Directions</h2> (F)
    <p>
      (G) You'll find us right in the center of
      downtown Webville. Come join us!
    </p>
  </body>
</html>
```

Notice how each tag in the HTML maps to what the browser displays.



Anatomia di un elemento HTML

`<p class="intro">Testo del paragrafo</p>`

Tag apertura

Attributo

Contenuto

Tag chiusura

I **tag** HTML sono usati per definire i marcatori (mark-up) di elementi HTML

Sono racchiusi tra parentesi angolari ("`<`" e "`>`") e contengono il nome dell'elemento

Di solito sono accoppiati, un esempio è dato da: `<p>` e `</p>`, detti rispettivamente **start tag** e **end tag**

Il testo tra start tag ed end tag è detto **contenuto** dell'elemento

Un documento HTML contiene quindi elementi composti da testo semplice delimitato da tag

Anatomia di un elemento HTML

`<p class="intro">Testo del paragrafo</p>`

Tag apertura

Attributo

Contenuto

Tag chiusura

Tag con contenuto

<code><h1>...</h1></code>	Titoli
<code><p>...</p></code>	Paragrafi
<code><a>...</code>	Link
<code><div>...</div></code>	Contenitori
...	

Tag auto-chiudenti (void)

<code>
</code>	A capo
<code><hr></code>	Linea orizzontale
<code></code>	Immagine
<code><input></code>	Campo input
...	

Grammatica poco rigorosa

HTML

- Ammette elementi senza chiusura come `<br>`
- I tag non sono cASe sEnSitive
- L'apertura e chiusura di tag annidati può essere “incrociata”
`<b><i>Testo corsivo grassetto</b></i>`

Esistono però delle buone pratiche che è bene rispettare e che diventano un obbligo in una versione più rigorosa del linguaggio chiamata XHTML

- Chiudere sempre anche i tag singoli: `<br></br>` o in forma sintetica `<br/>`
- Tag in minuscolo
- Apertura e chiusura senza incroci (in teoria non ammessi ma tollerati) `<b><i>...</i></b>`

Entity

HTML definisce un certo numero di entità (entity) per rappresentare i caratteri speciali (***escape character***) senza incorrere in problemi di codifica:

- Caratteri riservati a HTML (<, >, &, “, ecc.)
- Caratteri non presenti nell'ASCII a 7 bit

Result	Description	Entity Name	Entity Number
	non-breaking space	 	
<	less than	<	<
>	greater than	>	>
&	ampersand	&	&
¢	cent	¢	¢
£	pound	£	£
¥	yen	¥	¥
€	euro	€	€
©	copyright	©	©
®	registered trademark	®	®



Entity names are case sensitive.

La lista completa di escape characte HTML:

<http://www.theukwebdesigncompany.com/articles/entity-escape-characters.php>

Anatomia di un elemento HTML

```
<p class="intro">Testo del paragrafo</p>
```

Tag apertura

Attributo

Contenuto

Tag chiusura

Gli **attributi** sono informazioni aggiuntive che si inseriscono nel tag di apertura e che modificano il comportamento o le proprietà dell'elemento. Ogni attributo ha la forma `nome="valore"`, con il valore racchiuso tra virgolette (doppie o singole, ma per convenzione si usano le doppie).

Un elemento può avere più attributi separati da spazi. L'ordine degli attributi non ha importanza per il browser, ma per leggibilità si tende a mettere prima quelli più importanti.

Anatomia di un elemento HTML

`<p class="intro">Testo del paragrafo</p>`

Tag apertura

Attributo

Contenuto

Tag chiusura

Attributo	Descrizione	Esempio
<code>id</code>	Identificatore unico	<code>id="header"</code>
<code>class</code>	Classe CSS	<code>class="btn"</code>
<code>src</code>	Sorgente (immagini, script)	<code>src="img.png"</code>
<code>href</code>	URL di destinazione	<code>href="url.com"</code>
<code>alt</code>	Testo alternativo	<code>alt="Descrizione"</code>
<code>style</code>	Stile inline CSS	<code>style="color:red"</code>

Anatomia di un elemento HTML

```
<p class="intro">Testo del paragrafo</p>
```

Tag apertura

Attributo

Contenuto

Tag chiusura

Gli **attributi globali** possono essere usati su qualsiasi elemento HTML: tra i più comuni ci sono id, class, style...

Gli **attributi specifici** invece funzionano solo su determinati elementi: href è specifico di <a> e <link>, src è specifico di , <script> e <video> ...

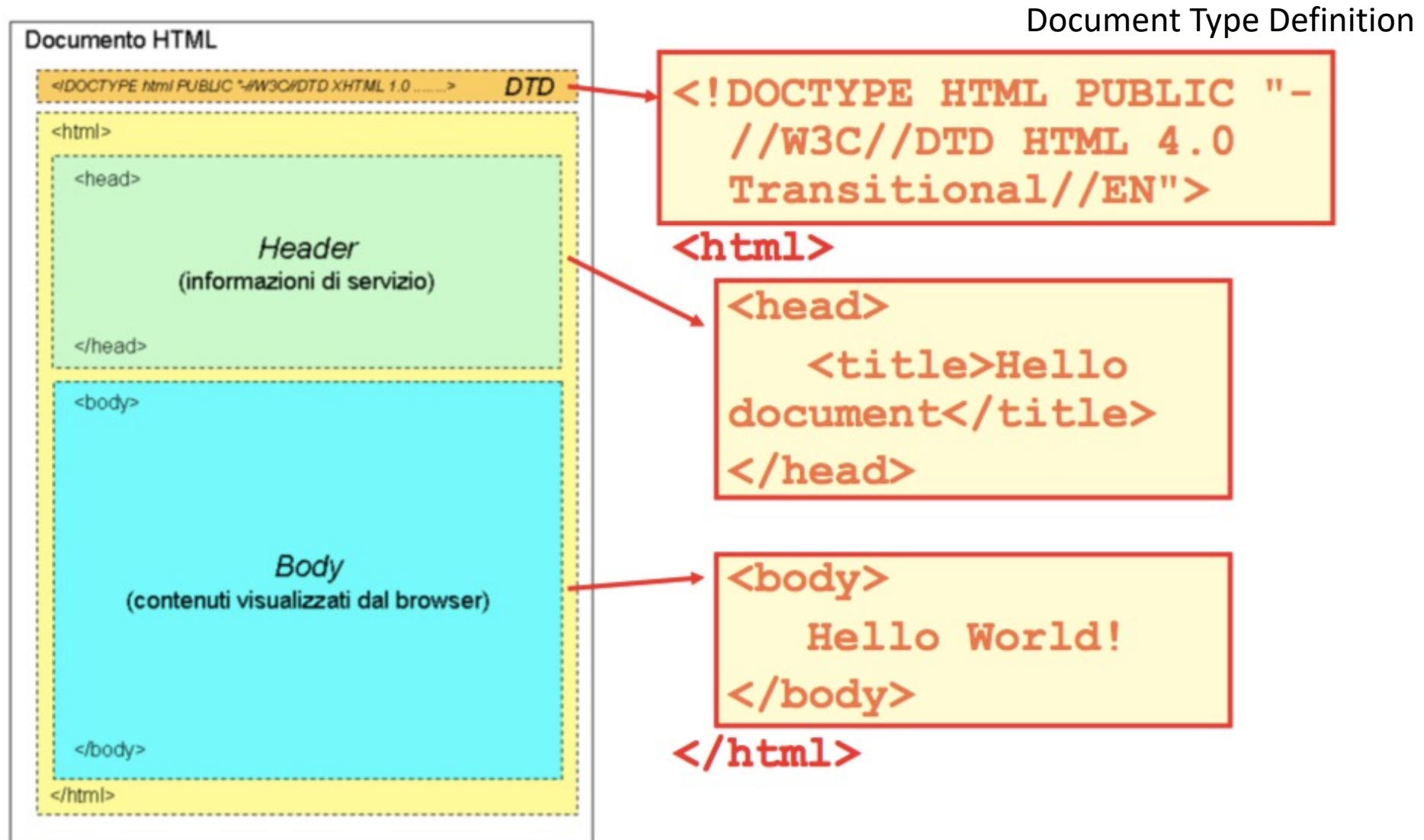
Gli **attributi booleani** invece non hanno bisogno di un valore: la loro semplice presenza nel tag indica "true", la loro assenza indica "false". Ad esempio, in <input type="text" required disabled>, gli attributi required e disabled sono booleani. È anche possibile scriverli nella forma required="required" o disabled="", ma la forma più concisa senza valore è quella preferita in HTML5.

Commenti

È possibile inserire commenti in qualunque punto all'interno di una pagina HTML con la seguente sintassi:

<!-- Questa stringa rappresenta un testo di commento -->

Struttura base di un documento HTML



Struttura base di un documento HTML

```
<html>
```

```
<head>
```

```
<title>Page title</title>
```

```
</head>
```

```
<body>
```

```
<h1>This is a heading</h1>
```

```
<p>This is a paragraph.</p>
```

```
<p>This is another paragraph.</p>
```

```
</body>
```

```
</html>
```



Only the `<body>` area (the white area) is displayed by the browser.

DTD Document Type Definition

Il primo elemento di un documento HTML è la definizione del tipo di documento (**Document Type Definition** o **DTD**)

Serve al browser per identificare le regole di interpretazione e visualizzazione da applicare al documento, es:

```
<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN"  
http://www.w3.org/TR/html4/loose.dtd>
```

È costituita da diverse parti:

- **HTML** il tipo di linguaggio utilizzato è l'HTML
- **PUBLIC** il documento è pubblico
- - le specifiche non sono registrate all'ISO (altrimenti +)
- **W3C** ente che ha rilasciato le specifiche
- **DTD HTML 4.01 Transitional**: versione di HTML
- **EN** la lingua con cui è scritta il DTD è l'inglese
- **http://...** URL delle specifiche

HTML 5 DOCTYPE

La vecchia versione del doctype per HTML 4.01:

```
<!DOCTYPE html PUBLIC "-//W3C//DTD HTML 4.01//EN"  
"http://www.w3.org/TR/html4/strict.dtd">
```

Il nuovo doctype per HTML5 è semplicemente:

```
<!doctype html>
```

Header

L'elemento `<head>` è la sezione invisibile di un documento HTML. Tutto ciò che si trova al suo interno non viene mostrato direttamente nella pagina web ma al suo interno si trovano esclusivamente metadati e riferimenti a risorse esterne.

Gli elemento che possono comparire nel `<head>` sono:

- **`<title>`** titolo della pagina
- **`<meta>`** i metadati del documento
- **`<base>`** URL di base per i link relativi
- **`<link>`** collegamenti a risorse esterne
- **`<script>`** codice eseguibile utilizzato dal documento (JavaScript)
- **`<style>`** informazioni di stile (CSS locali)

Header

Il tag **<title>** è l'unico elemento obbligatorio all'interno di <head>.

Definisce il titolo della pagina, che viene mostrato in tre contesti principali: nella tab del browser (la linguetta in alto), nei risultati dei motori di ricerca come snippet cliccabile, e nei segnalibri quando l'utente salva la pagina tra i preferiti.

```
<title>Introduzione a HTML – Corso di Sviluppo Web</title>
```

Header

Il tag **<base>** è un elemento void e definisce l'URL di base per tutti gli URL relativi presenti nel documento:

```
<base href="https://www.example.com/blog/" target="_blank">
```

Con questo tag, un link come

```
<a href="articolo.html">
```

verrà risolto come

```
https://www.example.com/blog/articolo.html
```

anziché essere relativo alla posizione del file corrente.

Header

I tag **<meta>** sono elementi void (senza tag di chiusura) che forniscono metadati sulla pagina. Non hanno un contenuto visibile, ma comunicano informazioni essenziali al browser, ai motori di ricerca e ai social network. Ogni **<meta>** utilizza una combinazione di attributi per definire il tipo di metadato e il suo valore.

Esistono due tipi di elementi meta, distinguibili dal primo attributo: **http-equiv** o **name**

Gli elementi di tipo *http-equiv* danno informazioni al browser su come gestire la pagina

<meta http-equiv=*nome* content=*valore*>

Gli elementi di tipo *name* forniscono informazioni utili ma non critiche

<meta nome=*nome* content=*valore*>

Header

<meta charset="UTF-8">

codifica caratteri. Va messo per primo. Senza di esso, accenti e simboli possono apparire corrotti.

<meta name="viewport" content="width=device-width, initial-scale=1.0">

indispensabile per il responsive design. Senza, i browser mobili rimpiccioliscono la pagina come fosse desktop.

<meta name="description" content="...">

snippet mostrato da Google nei risultati. Ideale: 150-160 caratteri.

<meta name="author" content="...">

autore del documento.

<meta name="robots" content="index, follow">

istruzioni per i crawler (noindex/nofollow per impedire l'indicizzazione).

<meta http-equiv="refresh" content="5;url=...">

redirect automatico dopo N secondi. Sconsigliato per redirect permanenti.

<meta name="theme-color" content="#0891B2">

colore della barra del browser su mobile.

Header

Il tag **<link>** è un elemento void che stabilisce una relazione tra il documento corrente e una risorsa esterna. L'attributo **rel** (relationship) definisce il tipo di relazione, mentre **href** indica l'URL della risorsa.

Fogli di stile CSS:

```
<link rel="stylesheet" href="style.css">
```

uso principale. È render-blocking: il browser non mostra la pagina finché il CSS non è scaricato.

Favicon:

```
<link rel="icon" href="/favicon.png" sizes="32x32">
```

icona nella tab del browser. Servono più dimensioni per diversi contesti (16x16, 32x32, 180x180 per Apple).

Header

Il tag **<style>** permette di inserire codice CSS direttamente all'interno del documento HTML, senza dover creare un file esterno.

Lo svantaggio è che il CSS interno non può essere messo in cache dal browser separatamente: viene riscaricato ogni volta che la pagina viene ricaricata, perché fa parte del documento HTML. Inoltre, non può essere condiviso tra pagine diverse dello stesso sito, portando a duplicazione del codice.

```
<style>
```

```
.hero {
```

```
    background-color: #0891b2;
```

```
    padding: 2rem;
```

```
}
```

```
</style>
```

Header

Il tag **<script>** è utilizzato per incorporare o collegare codice JavaScript.

Sebbene possa comparire anche nel `<body>`, la sua collocazione nel `<head>` è molto comune e ha implicazioni importanti sulle performance e sul comportamento della pagina.

Ci sono due modalità di utilizzo.

Lo **script esterno** collega un file JavaScript separato:

```
<script src="js/app.js"></script>
```

Lo **script inline** incorpora il codice direttamente nel documento:

```
<script>
```

```
    document.addEventListener('DOMContentLoaded', function() {  
        console.log('Pagina caricata!');  
    });
```

```
</script>
```

Header

`<title>`

Definisce il titolo della pagina, visualizzato nella barra del titolo del browser e nei risultati dei motori di ricerca. È obbligatorio in ogni documento HTML5 ed è cruciale per la SEO.

`<meta charset="UTF-8">`

Specifica la codifica dei caratteri. UTF-8 supporta tutti i caratteri Unicode e deve essere il primo elemento nell'head per evitare problemi di rendering.

`<meta name="viewport">`

Essenziale per il responsive design. `content="width=device-width, initial-scale=1.0"` assicura che la pagina si adatti alla larghezza del dispositivo.

`<meta name="description">`

Fornisce una descrizione della pagina usata dai motori di ricerca come snippet nei risultati. Lunghezza ideale: 150-160 caratteri.

`<link>`

Collega risorse esterne: fogli di stile CSS (`rel="stylesheet"`), favicon (`rel="icon"`), font web, e fogli di stile alternativi.

`<script>`

Include file JavaScript esterni (`src="..."`) o codice inline. L'attributo `defer` ritarda l'esecuzione fino al parsing completo dell'HTML.

Body

Il nesting (annidamento) è la tecnica di inserire elementi HTML all'interno di altri elementi, creando una struttura gerarchica ad albero. Una corretta indentazione del codice è essenziale per la leggibilità e la manutenibilità.

```
<div class="card">
  <h2>Titolo Card</h2>
  <p>Descrizione del contenuto.</p>
  <ul>
    <li>Primo</li>
    <li>Secondo</li>
  </ul>
  <a href="#">Link</a>
</div>
```

Regole fondamentali

- **Chiusura corretta:** i tag devono essere chiusi nell'ordine inverso di apertura (LIFO). `<b><i>OK</i></b>` è corretto, `<b><i>NO</b></i>` è errato.
- **Indentazione consistente:** usare 2 o 4 spazi (mai tab e spazi mescolati). Ogni livello di annidamento incrementa l'indentazione.
- **Attributi tra virgolette:** sempre usare le virgolette doppie per i valori degli attributi, anche quando non strettamente richiesto.
- **Tag in minuscolo:** per convenzione e compatibilità, scrivere tutti i tag e gli attributi in minuscolo.
- **Validare il codice:** utilizzare il validatore W3C (validator.w3.org) per verificare la correttezza sintattica.

Body

Il **<body>** può contenere centinaia di tipi diversi di elementi, che possiamo organizzare in grandi famiglie funzionali.

Intestazioni

Titoli h1-h6, organizzati in gerarchia

Strutture di testo

paragrafi, testo indentato, ecc.

Link e Multimedia

Ancore, immagini responsive, video, audio, canvas e SVG

Aspetto del testo

Grassetto corsivo ecc

Liste e Elenchi

Liste ordinate, non ordinate

Tabelle

Dati tabulati

Form e Input

tipi di input, validazione nativa, label, select, fieldset

Interattività e Script

Details, dialog, script, noscript, elementi deprecati

Semantica e Struttura

Header, nav, main, article, section, aside, footer, div/span

Heading <h1> - <h6>

I titoli (heading) definiscono la gerarchia informativa della pagina. HTML offre sei livelli, da <h1> (il più importante) a <h6> (il meno importante). I titoli **non servono solo a rendere il testo più grande**: hanno un valore semantico profondo. Gli screen reader per utenti non vedenti generano automaticamente un indice navigabile basato sui titoli, permettendo di saltare alla sezione desiderata. I **motori di ricerca attribuiscono un peso maggiore al testo nei titoli**, specialmente <h1> e <h2>, per comprendere il tema della pagina.

Regole fondamentali

- Un solo <h1> per pagina: rappresenta il titolo principale del contenuto.
- Mai saltare livelli: dopo <h2> si usa <h3>, non <h4>. La gerarchia deve riflettere la struttura logica.
- Mai usare heading per ingrandire il testo: per testo grande senza valore di intestazione, usa CSS font-size.

Heading `<h1>` - `<h6>`

Heading 1

Heading 2

Heading 3

Heading 3

Heading 4

Heading 4

Heading 2

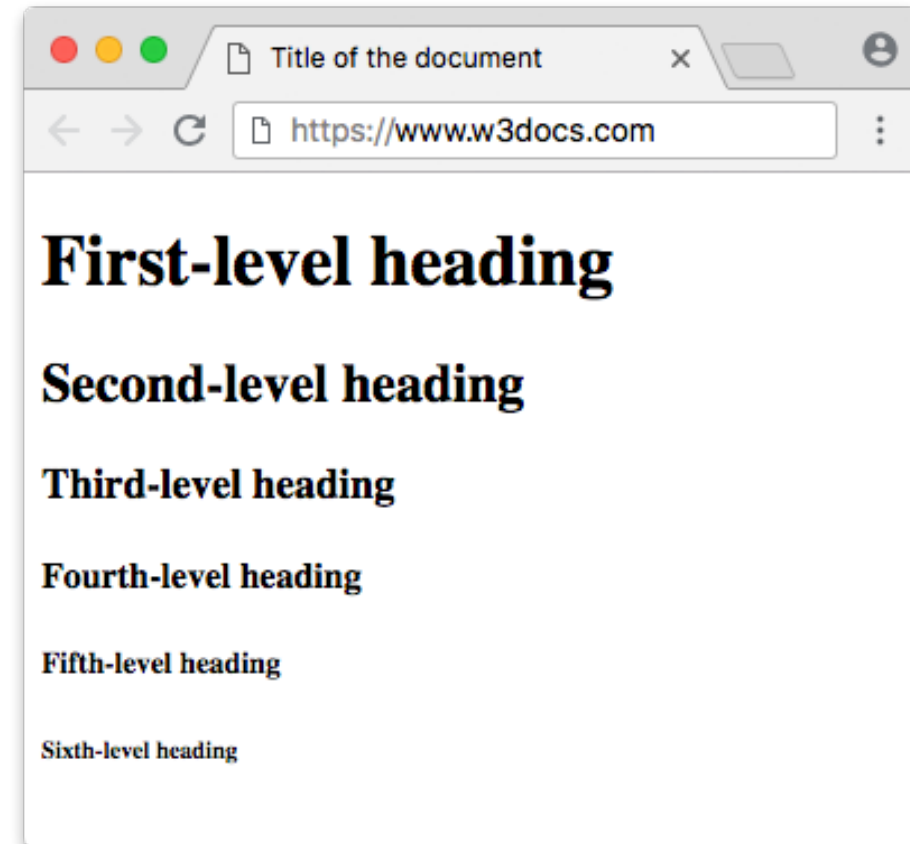
Heading 3

`<h1>First-level heading</h1>`

`<h2>Second-level heading</h2>`

`<h2>Third-level heading</h2>`

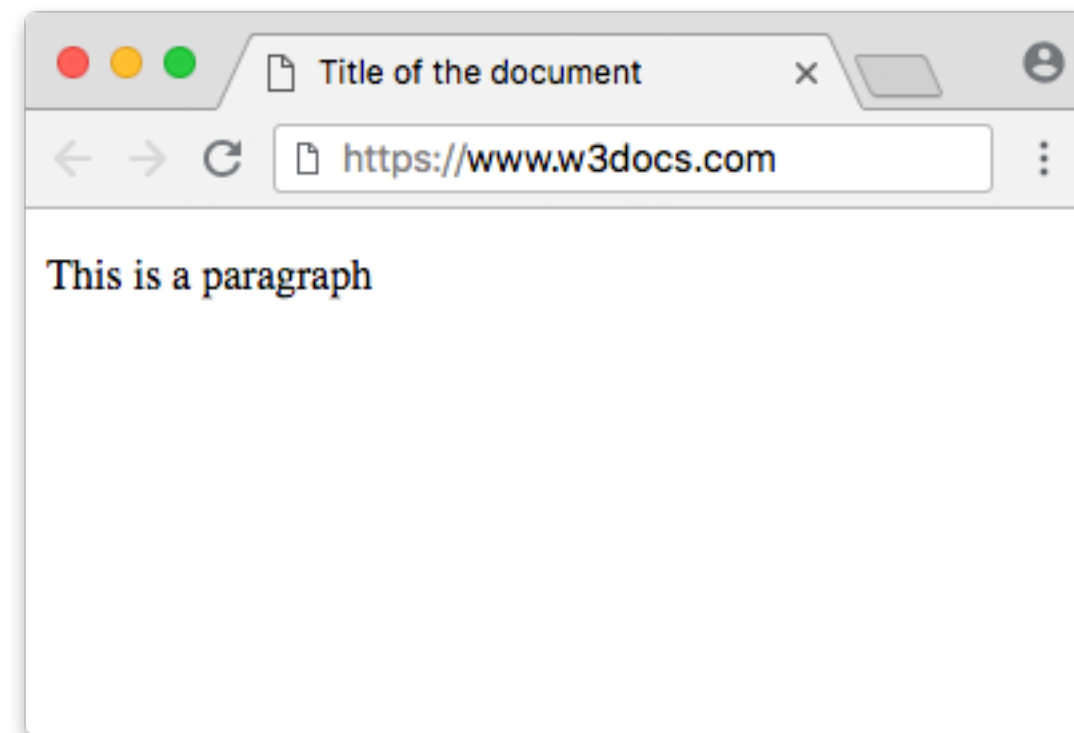
...



Paragrafi <p>

Il paragrafo <p> è l'elemento più utilizzato nel <body>. Definisce un blocco di testo con margini automatici. Il browser ignora a capo e spazi multipli nel sorgente. Per andare a capo serve
, ma solo per interruzioni semantiche (indirizzi, poesie), mai per spaziature tra blocchi.

<p>This is a paragraph</p>



Formattazione

Elemento	Tipo	Descrizione e Note
	Importanza semantica	Grassetto. Screen reader cambia tono. Contenuto importante.
	Grassetto presentazionale	Senza significato semantico. Per nomi prodotti, keyword.
	Enfasi semantica	Corsivo. Screen reader cambia intonazione. Indica enfasi.
<i>	Corsivo presentazionale	Per termini tecnici, parole straniere, titoli di opere.
<mark>	Evidenziazione	Sfondo giallo. Testo rilevante nel contesto corrente.
 / <ins>	Modifica documento	Testo rimosso (barrato) e inserito. Supportano datetime e cite.
<sub> / <sup>	Pedice e apice	Per formule chimiche (H ₂ O), notazioni matematiche (x ²).
<code>	Codice inline	Font monospazio. Frammenti di codice nel testo corrente.
<small>	Testo secondario	Note legali, copyright, disclaimer.

Formattazione

Elemento	Tipo
<code></code>	Importanza semantica
<code></code>	Grassetto presentazionale
<code></code>	Enfasi semantica
<code><i></code>	Corsivo presentazionale
<code><mark></code>	Evidenziazione
<code></code> / <code><ins></code>	Modifica documento
<code><sub></code> / <code><sup></code>	Pedice e apice
<code><code></code>	Codice inline
<code><small></code>	Testo secondario

This text is bold.

This text is strong.

This text is italic.

This text is emphasized.

HTML **Marked Formatting**

HTML Small Formatting

My favorite color is ~~blue~~ red.

My favorite color is red.

This is _{subscripted} text.

This is ^{superscripted} text.

Testo preformattato, Citazioni e Elementi speciali

`<pre>` e `<code>`

`<pre>` preserva spazi, tab e a capo esattamente come nel sorgente, in font monospazio. Fondamentale per codice, output, arte ASCII. `<pre><code>...</code></pre>` è la combinazione standard per listati multi-riga. `<code>` da solo è inline per frammenti nel testo.

`<blockquote>` e `<q>`

`<blockquote>` per citazioni lunghe da fonti esterne, con indentazione. L'attributo `cite` indica l'URL della fonte (metadato). `<q>` è inline per citazioni brevi: il browser aggiunge virgolette. `<cite>` indica titoli di opere (libro, film), NON persone.

`<abbr>`

Abbreviazioni con tooltip della forma estesa con mouse hover. `<abbr title="HyperText Markup Language">HTML</abbr>`

`<time>`

Date leggibili dalle macchine. L'attributo `datetime` contiene il formato ISO: `<time datetime="2025-03-15">15 marzo</time>`

`<details>` + `<summary>`

Widget espandibile/collassabile nativo senza JavaScript. L'attributo `open` lo rende espanso. Perfetto per FAQ e spoiler.

Il tag <a> anchor

L'elemento <a> (anchor) è il fondamento stesso del Web: senza i link.

L'attributo href (hypertext reference) specifica la destinazione e può contenere diversi tipi di URL.

URL assoluto (pagine esterne)

```
<a href="https://developer.mozilla.org">MDN</a>
```

URL relativo (pagine interne)

```
<a href="../index.html">Torna alla Home</a>
```

Ancora nella stessa pagina

```
<a href="#capitolo-3">Vai al Cap. 3</a>
```

Protocolli speciali mailto: → email. tel: → chiamata. sms: → messaggio.

```
<a href="mailto:info@uni.it?subject=Info">Email</a>
```

Il tag <a> anchor

```

<p>
  <a href="#section1">
    Introduzione</a><br>
  <a href="#section2">
    Concetti di base</a><br>
  <a href="#section2.1">
    Definizione del problema</a><br>
  ...
</p>

<h2><a name="section1">
  Introduzione</a></h2>
  ...sezione 1...

<h2><a name="section2">
  Concetti di base</a></h2>
  ...sezione 2...

<h3><a name="section2.1">
  Definizione del problema</a></h3>
  ...sezione 2.1...
  
```



Sommario

[Introduzione](#)
[Concetti di base](#)
[Definizione del problema](#)

...

Introduzione

...sezione 1...

Concetti di base

...sezione 2...

Definizione del problema

...sezione 2.1...

Il tag <a> anchor

Si può esprimere un'ancora di destinazione in forma "implicita", cioè senza utilizzare il tag <a>

- È sufficiente assegnare l'attributo **ID** a un qualunque elemento della pagina

```

<p>
  <a href="#section1">
    Introduzione</a><br>
  <a href="#section2">
    Concetti di base</a><br>
  <a href="#section2.1">
    Definizione del problema</a><br>
  ...
</p>

<h2 id="section1">
  Introduzione</h2>
  ...sezione 1...

<h2 id="section2">
  Concetti di base</h2>
  ...sezione 2...

<h3 id="section2.1">
  Definizione del problema</h3>
  ...sezione 2.1...
  
```

Sommario

[Introduzione](#)
[Concetti di base](#)
[Definizione del problema](#)

...

Introduzione

...sezione 1...

Concetti di base

...sezione 2...

Definizione del problema

...sezione 2.1...

Il tag `<a>` anchor

URL assoluti

URL che iniziano con `http://` `<a href="http://otherhost.com/a/b.html">...</a>`

- Interpretati indipendentemente dall'URL della pagina corrente

URL relativi

URL che iniziano con `.` `<a href="./g.html">...</a>`

- Interpretati rispetto alla "directory" del file corrente

URL che iniziano con `..` `<a href="../h.html">...</a>`

- Interpretati rispetto alla directory principale del file corrente

- URL che iniziano con `/` `<a href="/i.html">...</a>`

Interpretati rispetto al nome host dell'URL

Il tag <a> anchor

http://somehost.com/a/b/c/d/e/f.html

Link	Risultato
...	http://otherhost.com/a/b.html
... <u>or</u> ...	http://somehost.com/a/b/c/d/e/g.html
...	http://somehost.com/a/b/c/d/h.html
...	http://somehost.com/i.html

oltre a <a href...>, questi URL possono essere applicati a img, form, script, style e ad altri elementi.

Il tag `<a>` anchor

L'attributo `target` specifica dove aprire il documento collegato

```
<a href="http://www.w3schools.com/" target="_blank">Visit W3Schools!</a>
```

Target Value	Description
<code>_blank</code>	Opens the linked document in a new window or tab
<code>_self</code>	Opens the linked document in the same frame as it was clicked (this is default)
<code>_parent</code>	Opens the linked document in the parent frame
<code>_top</code>	Opens the linked document in the full body of the window
<code>framename</code>	Opens the linked document in a named frame

Il tag Image

Il tag viene utilizzato per incorporare un'immagine in una pagina HTML. Tecnicamente, le immagini non vengono inserite direttamente in una pagina web, ma collegate ad essa. Il tag crea uno spazio contenitore per l'immagine a cui fa riferimento.

ha due attributi obbligatori:

- **src:** Specifica il percorso dell'immagine
- **alt:** Specifica un testo alternativo per l'immagine.

```

```

Nota: è inoltre fondamentale specificare sempre la larghezza e l'altezza dell'immagine. Se queste non vengono specificate, la pagina potrebbe sfarfallare durante il caricamento dell'immagine.

Il tag Image

```

```

Formati immagine per il web

Formato	Uso ideale	Note
JPEG	Fotografie, immagini complesse	Compressione lossy, no trasparenza
PNG	Grafiche, loghi, screenshot	Lossless, supporta trasparenza alpha
GIF	Animazioni semplici, icone	Max 256 colori, animazione nativa
SVG	Icone, loghi, illustrazioni	Vettoriale, scalabile senza perdita
WebP	Uso generale moderno	Migliore compressione, trasparenza

Il tag Image

Attribute	Value	Description
<u>alt</u>	<i>text</i>	Specifies an alternate text for an image
crossorigin	anonymous use-credentials	Allow images from third-party sites that allow cross-origin access to be used with canvas
<u>height</u>	<i>pixels</i>	Specifies the height of an image
<u>ismap</u>	ismap	Specifies an image as a server-side image map
<u>loading</u>	eager lazy	Specifies whether a browser should load an image immediately or to defer loading of images until some conditions are met
<u>longdesc</u>	<i>URL</i>	Specifies a URL to a detailed description of an image
<u>referrerpolicy</u>	no-referrer no-referrer-when-downgrade origin origin-when-cross-origin unsafe-url	Specifies which referrer information to use when fetching an image
sizes	<i>sizes</i>	Specifies image sizes for different page layouts
<u>src</u>	<i>URL</i>	Specifies the path to the image
srcset	<i>URL-list</i>	Specifies a list of image files to use in different situations
<u>usemap</u>	<i>#mapname</i>	Specifies an image as a client-side image map
<u>width</u>	<i>pixels</i>	Specifies the width of an image

srcset e sizes

srcset elenca immagini con larghezza reale in pixel (descrittore w). sizes indica lo spazio occupato a diverse dimensioni di viewport. Il browser sceglie la versione più adatta in base a schermo e densità pixel. Es: srcset="foto-400.jpg 400w, foto-800.jpg 800w, foto-1600.jpg 1600w" sizes="(max-width: 600px) 100vw, 50vw"

Loading

Specifica se un browser deve caricare un'immagine immediatamente o se deve posticipare il caricamento delle immagini fino a quando non vengono soddisfatte determinate condizioni.

Il tag Image

usemap specifica un'immagine come mappa immagine (una mappa immagine è un'immagine con aree cliccabili).

```
<!DOCTYPE html>
<html>
<body>

<h1>The map and area elements</h1>

<p>Click on the computer, the phone, or the cup of coffee to go to a new page and read more about the topic:</p>



<map name="workmap">
  <area shape="rect" coords="34,44,270,350" alt="Computer" href="computer.htm">
  <area shape="rect" coords="290,172,333,250" alt="Phone" href="phone.htm">
  <area shape="circle" coords="337,300,44" alt="Cup of coffee" href="coffee.htm">
</map>

</body>
</html>
```

computer.htm

phone.htm

coffee.htm



<video> e <audio>

Prima di HTML5, incorporare audio e video nelle pagine web richiedeva plugin esterni come Flash Player. HTML5 introduce i tag nativi <audio> e <video> che funzionano direttamente nel browser senza plugin aggiuntivi, con supporto per controlli integrati e API JavaScript.



Il tag <video>

```
<video controls width="640"
      poster="thumb.jpg">
  <source src="video.mp4"
        type="video/mp4">
</video>
```

Attributi: controls (mostra controlli), autoplay, muted, loop, poster (immagine di anteprima), preload (none/metadata/auto).

Formati: MP4 (H.264) per massima compatibilità, WebM per Chrome/Firefox.



Il tag <audio>

```
<audio controls>
  <source src="audio.mp3"
        type="audio/mpeg">
</audio>
```

Stessi attributi di <video> eccetto poster e width/height.

Formati: MP3 (universale), OGG Vorbis, WAV (non compresso).

Il tag <source> multiplo permette di specificare formati alternativi — il browser sceglie il primo che supporta.

Liste e elenchi

Le liste sono elementi fondamentali per organizzare informazioni in modo strutturato. HTML supporta tre tipi di liste: non ordinate (unordered), ordinate (ordered) e di definizione (definition). Possono essere annidate per creare strutture gerarchiche complesse.

Lista non ordinata

```
<ul>
  <li>Mela</li>
  <li>Banana</li>
  <li>Arancia</li>
</ul>
```

Elenchi di elementi senza ordine specifico. Il browser mostra un bullet point per ogni . Personalizzabile con CSS (list-style-type).

Lista ordinata

```
<ol>
  <li>Primo</li>
  <li>Secondo</li>
  <li>Terzo</li>
</ol>
```

Elenchi con ordine significativo (procedure, classifiche). Numerazione automatica. Attributi: start, reversed, type (1, a, A, i, I).

Lista definizione <dl>

```
<dl>
  <dt>HTML</dt>
  <dd>Linguaggio di
      markup</dd>
</dl>
```

Coppie termine-definizione. <dt> per il termine, <dd> per la definizione. Ideale per glossari, FAQ, metadati.

Liste e elenchi

Le liste sono elementi fondamentali per organizzare informazioni in modo strutturato. HTML supporta tre tipi di liste: non ordinate (unordered), ordinate (ordered) e di definizione (definition). Possono essere annidate per creare strutture gerarchiche complesse.

Lista non ordinata

```
<ul>
  <li>Mela</li>
  <li>Banana</li>
  <li>Arancia</li>
</ul>
```

Elenchi di elementi senza ordine specifico. Il browser mostra un bullet point per ogni . Personalizzabile con CSS (list-style-type).

Lista ordinata

```
<ol>
  <li>Primo</li>
  <li>Secondo</li>
  <li>Terzo</li>
</ol>
```

Elenchi con ordine significativo (procedure, classifiche). Numerazione automatica. Attributi: start, reversed, type (1, a, A, i, I).

Lista definizione <dl>

```
<dl>
  <dt>HTML</dt>
  <dd>Linguaggio di
      markup</dd>
</dl>
```

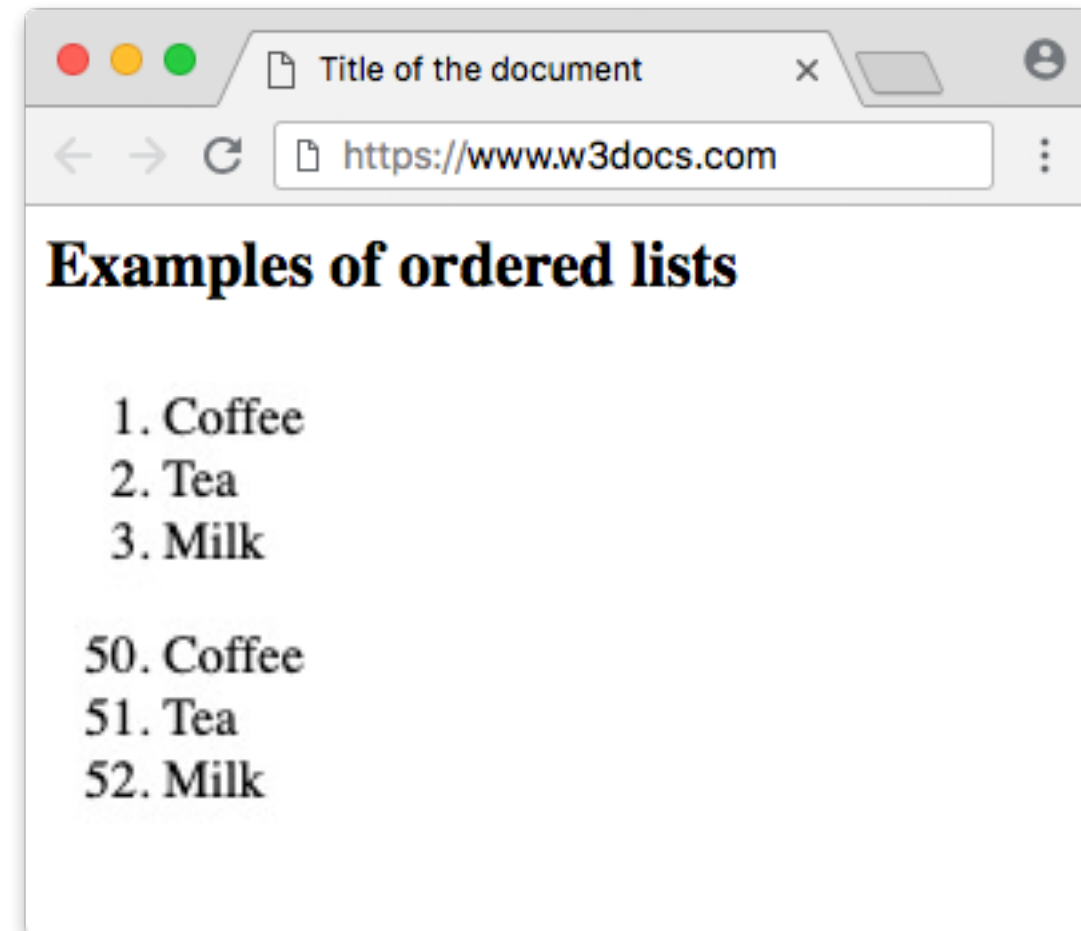
Coppie termine-definizione. <dt> per il termine, <dd> per la definizione. Ideale per glossari, FAQ, metadati.

Liste e elenchi

Lista ordinata

```
<body>
  <h2>Examples of ordered lists</h2>
  <ol>
    <li>Coffee</li>
    <li>Tea</li>
    <li>Milk</li>
  </ol>

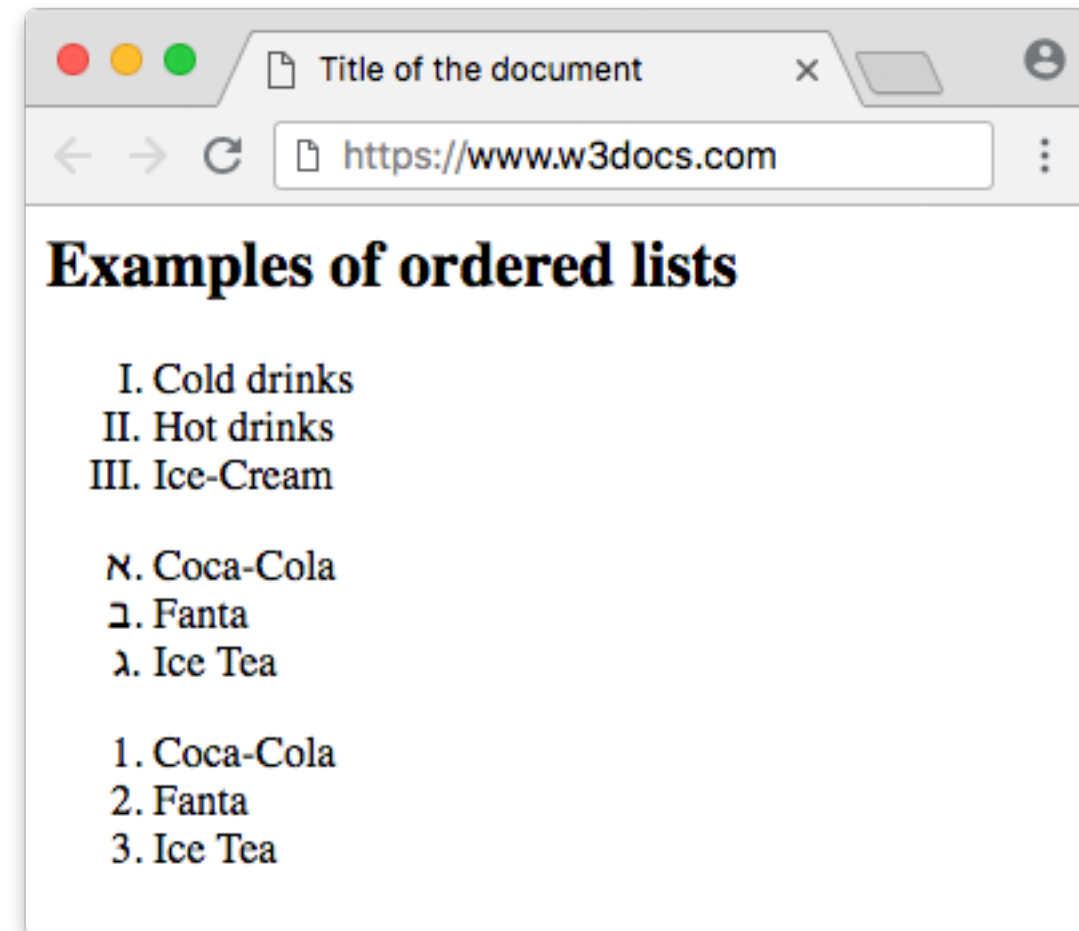
  <ol start="50">
    <li>Coffee</li>
    <li>Tea</li>
    <li>Milk</li>
  </ol>
</body>
```



Liste e elenchi

Lista ordinata

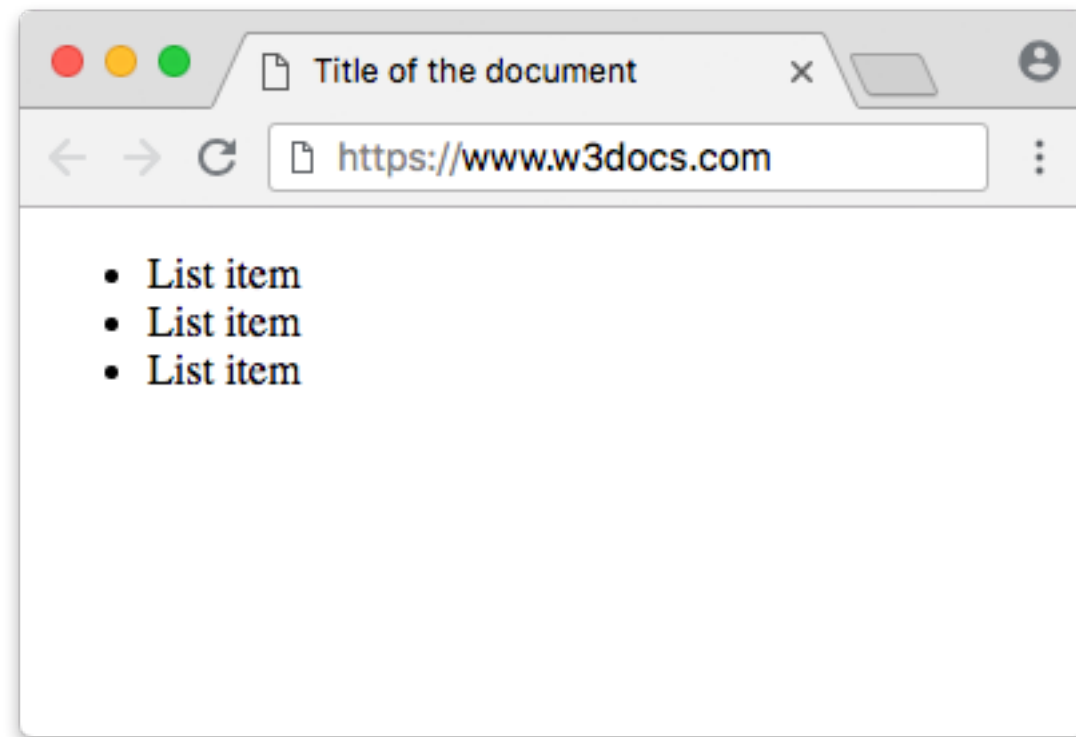
```
<body>
  <h2>Examples of ordered lists</h2>
  <ol style="list-style-type: upper-roman">
    <li>Cold drinks</li>
    <li>Hot drinks</li>
    <li>Ice-Cream</li>
  </ol>
  <ol style="list-style-type: hebrew">
    <li>Coca-Cola</li>
    <li>Fanta</li>
    <li>Ice Tea</li>
  </ol>
  <ol>
    <li>Coca-Cola</li>
    <li>Fanta</li>
    <li>Ice Tea</li>
  </ol>
</body>
```



Liste e elenchi

Lista non ordinata

```
<body>
  <ul>
    <li>List item</li>
    <li>List item</li>
    <li>List item</li>
  </ul>
</body>
```



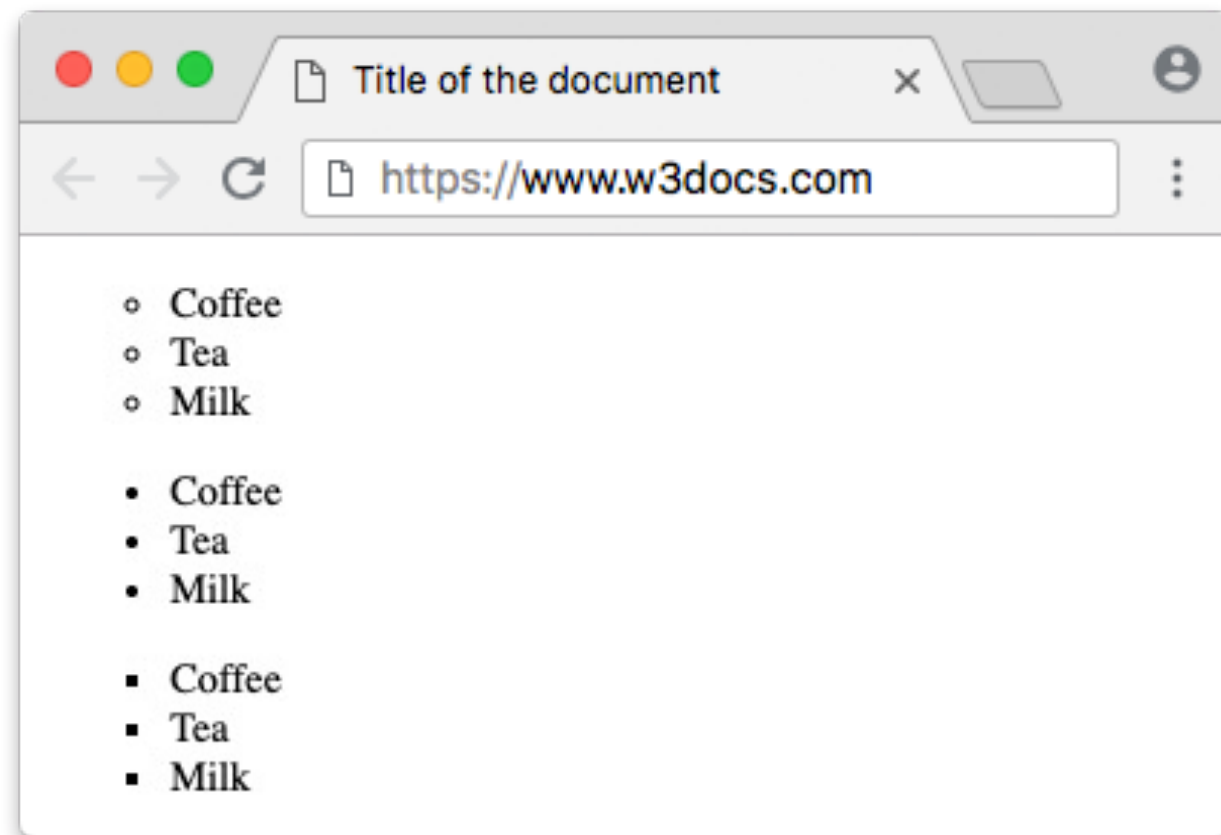
Liste e elenchi

Lista non ordinata

```
<body>
  <ul style="list-style-type:circle">
    <li>Coffee</li>
    <li>Tea</li>
    <li>Milk</li>
  </ul>

  <ul style="list-style-type:disc">
    <li>Coffee</li>
    <li>Tea</li>
    <li>Milk</li>
  </ul>

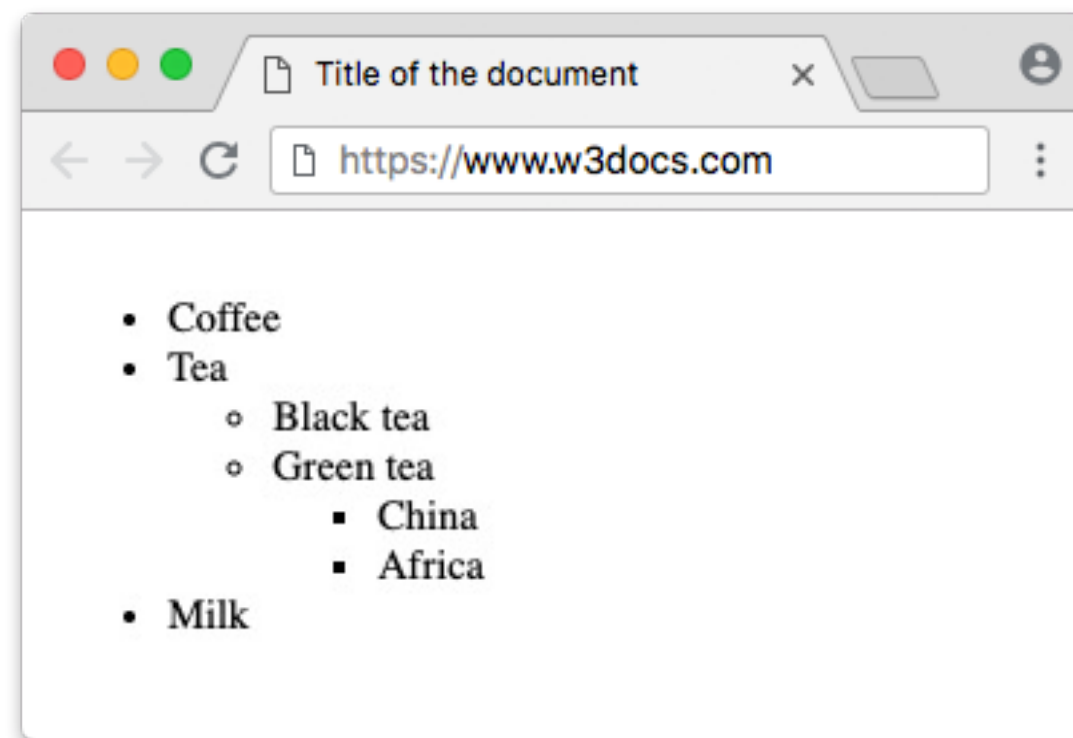
  <ul style="list-style-type:square">
    <li>Coffee</li>
    <li>Tea</li>
    <li>Milk</li>
  </ul>
</body>
```



Liste e elenchi

Lista non ordinata

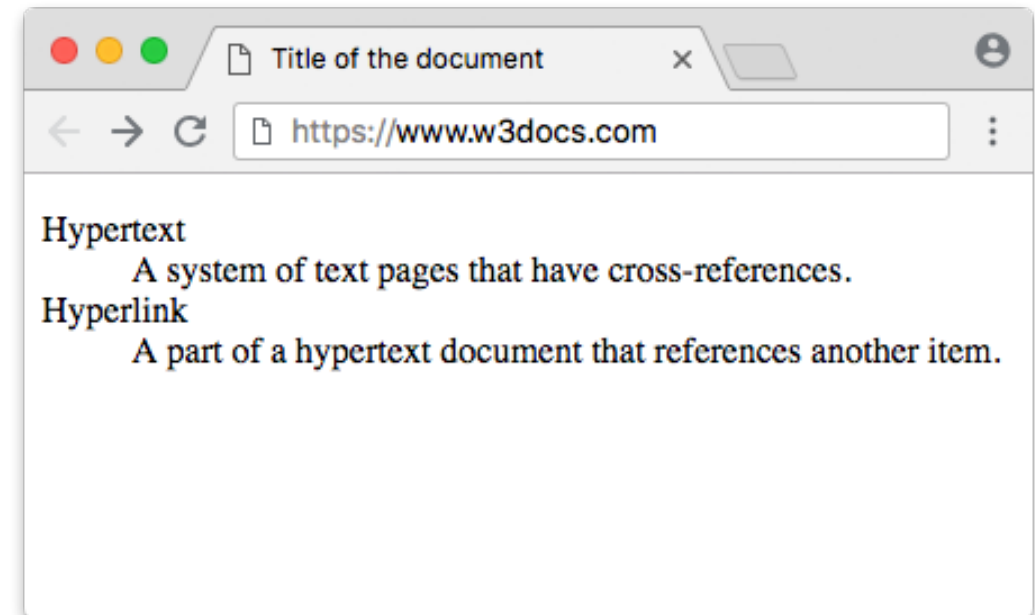
```
<body>
  <ul>
    <li>Coffee</li>
    <li>Tea
      <ul>
        <li>Black tea</li>
        <li>Green tea
          <ul>
            <li>China</li>
            <li>Africa</li>
          </ul>
        </li>
      </ul>
    </li>
    <li>Milk</li>
  </ul>
</body>
```



Liste e elenchi

Lista definizione <dl>

```
<body>
  <dl>
    <dt>Hypertext</dt>
    <dd>A system of text pages that have cross-references.</dd>
    <dt>Hyperlink</dt>
    <dd>A part of a hypertext document that references another item.</dd>
  </dl>
</body>
```



Tabelle

Le tabelle in HTML servono esclusivamente per presentare dati tabulari (mai per il layout della pagina, pratica obsoleta). Una tabella ben strutturata utilizza `<thead>`, `<tbody>`, `<tfoot>` per separare intestazione, corpo e piede, migliorando accessibilità e manutenibilità.

```
<table>
  <caption>Voti Esami</caption>
  <thead>
    <tr>
      <th>Materia</th>
      <th>Voto</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <td>TSW</td>
      <td>18</td>
    </tr>
  </tbody>
</table>
```

Elementi della tabella

<code><table></code>	Contenitore principale della tabella
<code><caption></code>	Titolo descrittivo della tabella (accessibilità)
<code><thead></code>	Raggruppa le righe di intestazione
<code><tbody></code>	Raggruppa le righe di dati
<code><tfoot></code>	Raggruppa le righe di piede (totali, medie)
<code><tr></code>	Table Row — definisce una riga
<code><th></code>	Table Header — cella di intestazione (grassetto)
<code><td></code>	Table Data — cella dati standard
<code>colspan/rowspan</code>	Unione di celle orizzontale/verticale

Elementi blocco, elementi inline

In HTML ogni elemento ha un comportamento di visualizzazione predefinito che determina come occupa lo spazio nella pagina e come interagisce con gli elementi circostanti. Questa distinzione fondamentale divide gli elementi in due grandi categorie: **elementi di blocco** (block-level) ed **elementi inline** (inline-level).

Un **elemento di blocco** si comporta come un "mattoncino" nella costruzione della pagina.

- inizia sempre su una nuova riga, indipendentemente da quanto spazio sia disponibile sulla riga precedente.
- si espande automaticamente per occupare tutta la larghezza disponibile del suo contenitore genitore.
- Può contenere altri elementi blocco o anche elementi inline

Un **elemento inline** si comporta come una parola all'interno di una frase.

- non inizia su una nuova riga, ma si inserisce nel flusso del testo esattamente dove appare nel codice, uno accanto all'altro sulla stessa riga.
- occupa solo lo spazio necessario al suo contenuto, senza espandersi. Un `<strong>` largo 50 pixel occuperà esattamente 50 pixel, lasciando il resto della riga disponibile per gli elementi successivi.
- Può contenere solo altri elementi inline

Elementi blocco, elementi inline

In HTML ogni elemento ha un comportamento di visualizzazione predefinito che determina come occupa lo spazio nella pagina e come interagisce con gli elementi circostanti. Questa distinzione fondamentale divide gli elementi in due grandi categorie: **elementi di blocco** (block-level) ed **elementi inline** (inline-level).

Un **elemento di blocco** si comporta come un "mattoncino" nella costruzione della pagina.

<code><address></code>	<code><article></code>	<code><aside></code>	<code><blockquote></code>	<code><canvas></code>	<code><dd></code>	<code><div></code>
<code><dl></code>	<code><dt></code>	<code><fieldset></code>	<code><figcaption></code>	<code><figure></code>	<code><footer></code>	<code><form></code>
<code><h1>–<h6></code>	<code><header></code>	<code><hr></code>	<code></code>	<code><main></code>	<code><nav></code>	<code><noscript></code>
<code></code>	<code><p></code>	<code><pre></code>	<code><section></code>	<code><table></code>	<code><tfoot></code>	<code></code>
<code><video></code>						

Un **elemento inline** si comporta come una parola all'interno di una frase.

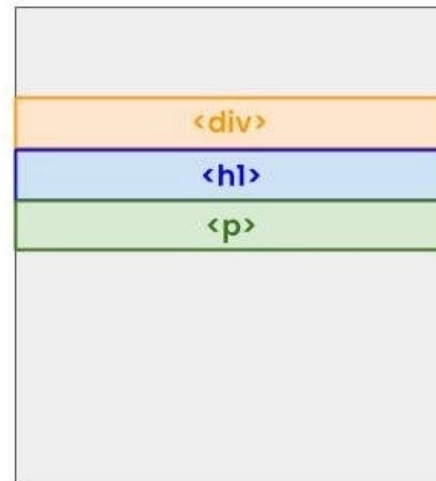
<code><a></code>	<code><abbr></code>	<code><acronym></code>	<code></code>	<code><bdo></code>	<code><big></code>	<code>
</code>
<code><button></code>	<code><cite></code>	<code><code></code>	<code><dfn></code>	<code></code>	<code><i></code>	<code></code>
<code><input></code>	<code><kbd></code>	<code><label></code>	<code><map></code>	<code><object></code>	<code><output></code>	<code><q></code>
<code><samp></code>	<code><script></code>	<code><select></code>	<code><small></code>	<code></code>	<code></code>	<code><sub></code>
<code><sup></code>	<code><textarea></code>	<code><time></code>	<code><tt></code>	<code><var></code>		

Elementi blocco, elementi inline

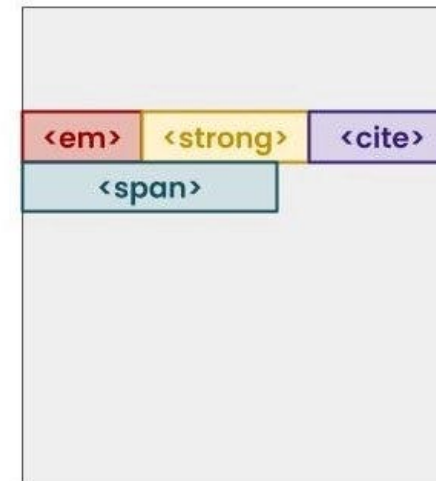
In HTML ogni elemento ha un comportamento di visualizzazione predefinito che determina come occupa lo spazio nella pagina e come interagisce con gli elementi circostanti. Questa distinzione fondamentale divide gli elementi in due grandi categorie: **elementi di blocco** (block-level) ed **elementi inline** (inline-level).

HTML ELEMENT

BLOCK-LEVEL



INLINE



Elementi Deprecati

Diversi elementi del <body> sono stati deprecati perché mescolavano struttura e presentazione. La regola moderna: HTML per struttura, CSS per presentazione.

❌ Deprecati — da NON usare

<center>	→ CSS text-align: center
	→ CSS font-family, color, font-size
<marquee>	→ Mai standardizzato
<blink>	→ Testo lampeggiante (famigerato)
<frame>/<frameset>	→ CSS Grid/Flexbox per layout

✓ <iframe>: Ancora in uso

```
<iframe  
src="https://youtube.com/embed/..."  
width="560" height="315"  
title="Video" loading="lazy"  
allowfullscreen></iframe>
```

Incorpora documento esterno: video YouTube,
Google Maps, widget, chat.

Principio guida: se usi un attributo HTML per cambiare aspetto visuale (allineamento, colore, dimensione, bordo), fermati e usa CSS. **HTML = struttura e significato. CSS = presentazione.**

Validazione

La validazione del codice HTML verifica che il documento rispetti le specifiche ufficiali del W3C. Un codice valido garantisce compatibilità cross-browser, migliore accessibilità, indicizzazione SEO ottimale e manutenibilità a lungo termine. Errori di validazione possono causare comportamenti imprevedibili del rendering.

Il validatore ufficiale (validator.w3.org) permette di verificare il codice HTML inserendo un URL, caricando un file o incollando il codice sorgente. Segnala errori (violations) e avvisi (warnings) con spiegazioni dettagliate e link alla specifica.

Errori di validazione più comuni

- Tag non chiusi: un `<div>` aperto senza il corrispondente `</div>` causa problemi di layout e nesting errato.
- Tag annidati in modo errato: `<b><i>testo</b></i>` viola la regola LIFO — corretto: `<b><i>testo</i></b>`.
- Attributi duplicati: lo stesso attributo non può apparire due volte nello stesso tag.
- ID duplicati: ogni valore di id deve essere unico nell'intero documento HTML.
- DOCTYPE mancante: senza `<!DOCTYPE html>`, il browser entra in "quirks mode" con comportamenti imprevedibili.



Check the markup (HTML, XHTML, ...) of Web documents

Validate by URI

Validate a document online:

Address: http://www.w3cschools.com/html/demo_html.asp

► More Options

Check

This validator checks the [markup validity](#) of Web documents in HTML, XHTML, SMIL, MathML, etc. If you wish to validate specific content such as [RSS/Atom feeds](#) or [CSS stylesheets](#), [MobileOK content](#), or to [find broken links](#), there are [other validators and tools](#) available. As an alternative you can also try our [non-DTD-based validator](#).

The W3C validators are developed with assistance from the Mozilla Foundation, and supported by community donations.

[Donate](#) and help us build better tools for a better web.

6088

Validazione

1. **Info** The Content-Type was `text/html`. Using the HTML parser.
2. **Info** Using the schema for HTML with SVG 1.1, MathML 3.0, RDFa 1.1, and ITS 2.0 support.
3. **Error** An `img` element must have an `alt` attribute, except under certain conditions. For details, consult [guidance on providing text alternatives for images](#).

[From line 6, column 63; to line 6, column 84](#)

```
e</h1> <p>↵Join
```

Devi semplicemente aggiungere un attributo alt agli elementi `<img>` in HTML5.

References

W3Schools

[w3schools.com](https://www.w3schools.com)

Tutorial interattivi con editor online "Try it Yourself". Ottimo per esercitarsi rapidamente con i tag HTML e vedere il risultato in tempo reale.

W3C Specification

html.spec.whatwg.org

La specifica ufficiale del linguaggio HTML (Living Standard WHATWG). Riferimento definitivo per la semantica e il comportamento di ogni elemento.

Can I Use

caniuse.com

Database che mostra il supporto di ogni feature HTML/CSS/JS nei diversi browser. Essenziale per verificare la compatibilità cross-browser.

CodePen / JSFiddle

codepen.io / jsfiddle.net

Ambienti di sviluppo online per sperimentare con HTML, CSS e JS. Perfetti per prototipare velocemente e condividere esempi di codice.

MDN Web Docs (Mozilla) developer.mozilla.org

La risorsa più completa e aggiornata per HTML, CSS e JavaScript. Documentazione ufficiale con esempi interattivi, guide e tutorial per ogni livello.

HTML 4.01

www.w3.org/TR/html401

La specifica del linguaggio HTML 4.01 Raccomandazione del W3C del 24 dicembre 1999, sostituita il 27 marzo 2018.